

Методы бизнес-логики

Метод — это функция (процедура), при помощи которой осуществляется взаимодействие как с данными БД (чтение, создание, удаление и другие операции с записями), так и с другими сущностями, например, полученными из сторонних сервисов. Все методы объявляют в [объектах БЛ](#).

Реализации методов бизнес-логики:

- Python
- Встроенная функция
- SQL-запрос.

Типы методов бизнес-логики:

- [Автогенерируемые](#)
- [Списочные](#):
 - [Декларативный](#)
 - [Реализуемый вручную](#)
- Методы [работы с файлами](#):
 - [Прочитать \(Read file method\)](#),
 - [Загрузить \(Upload file method\)](#),
 - [Загрузить с прикреплением \(Upload and attach file method\)](#)
 - [Список по UUID \(File UUID list method\)](#)
- [Удалённые вызовы](#):
 - [Proху метод](#)
 - [HTTP запрос](#)
- [Произвольный метод](#)
- [Область видимости](#)
- [Обработчик асинхронного вызова](#)

Источник данных в контексте сервисного фреймворка - это метод или набор методов сервиса. Для того, чтобы с ним мог работать стандартный компонент на UI, у таких методов API должен отвечать стандартному [контракту](#).

Для подключения методов данных произвольного контракта необходимо создать компонент-адаптер.

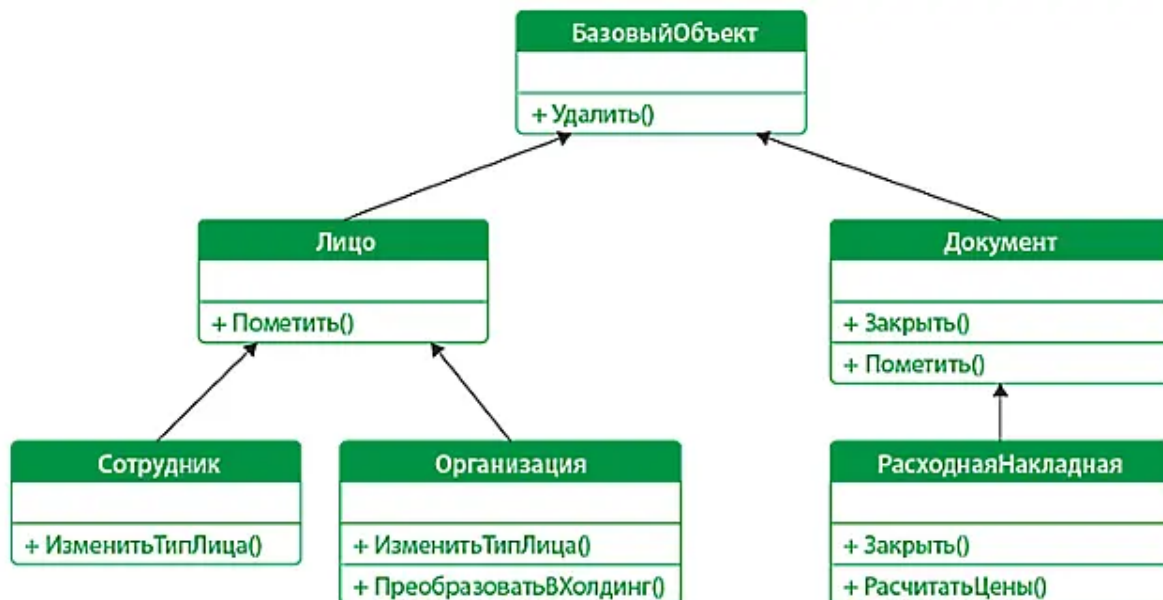
Наследование методов

Объекты бизнес-логики формируют иерархическую структуру, визуально отображаемую в [справочнике объектов](#).

Верхний уровень иерархии в любом справочнике всегда занимает объект с названием "БазовыйОбъект" ("Object"). Он создаётся по умолчанию, но визуально в интерфейсе не отображён. От него наследуются все остальные объекты.

Структура наследования объектов определяет наследование методов этих объектов. Методы объектов верхнего иерархического уровня наследуются объектами нижнего уровня. Если метод не описан у данного объекта, но есть у его предка, то вызывается метод предка.

Например, для структуры справочника объектов следующего вида:



вызов метода "Сотрудник.Пометить()" должен вызвать в итоге метод "Лицо.Пометить()".

Таким образом, у объекта "Сотрудник" будет три метода:

- "ИзменитьТипЛица",
- "Пометить",
- "Удалить".

Такое наследование, во-первых, позволит клиенту сразу видеть все методы объекта "Сотрудник", а во-вторых, позволит в дальнейшем реализовать на сервере метод "Сотрудник.Пометить()" без изменения вызова в клиентах.

Порядок наследования устанавливается и для [обработчиков методов БЛ](#).

При наследовании одноименных методов автогенерируемые методы будут иметь приоритет над методами с произвольным количеством параметров (Custom method).

Сами автогенерируемые методы не наследуются.

Механизм перекрытия методов по имени

Для одноименных методов, описанных в разных модулях реализован механизм перекрытия по имени. При одновременном выполнении условий:

- в Модуль1 описан метод бизнес-логики Объект.Метод и в Модуль2 описан метод бизнес-логики Объект.Метод, с тем же числом аргументов,
- Модуль2 по зависимостям загружается после Модуль1,

будет использоваться реализация из Модуль2.

Аналогично перекрываются одноименные [обработчики](#) одного метода бизнес-логики, описанные в разных модулях.

В **одном и том же объекте или его наследниках** крайне нежелательно одновременно объявлять с **одинаковыми именами** списочный метод (с четырьмя параметрами) и любой иной тип метода с одним параметром. По историческим причинам списочные методы регистрируются **дважды: с четырьмя и одним параметром**.

Если списочный метод имеет [доступность](#) Service Contract, а иной **одноименный с ним** метод - Internal Application Contract или более закрытую, в логах облака будут регистрироваться ошибки при обращении к списочному методу вида:

"Вызов метода, не являющегося частью публичного API сервиса.

...

Вызов метода с доступностью "Internal Application Contract" допустим только между сервисами одного приложения."

На объекты, которые не связаны наследованием между собой, данная особенность не распространяется.

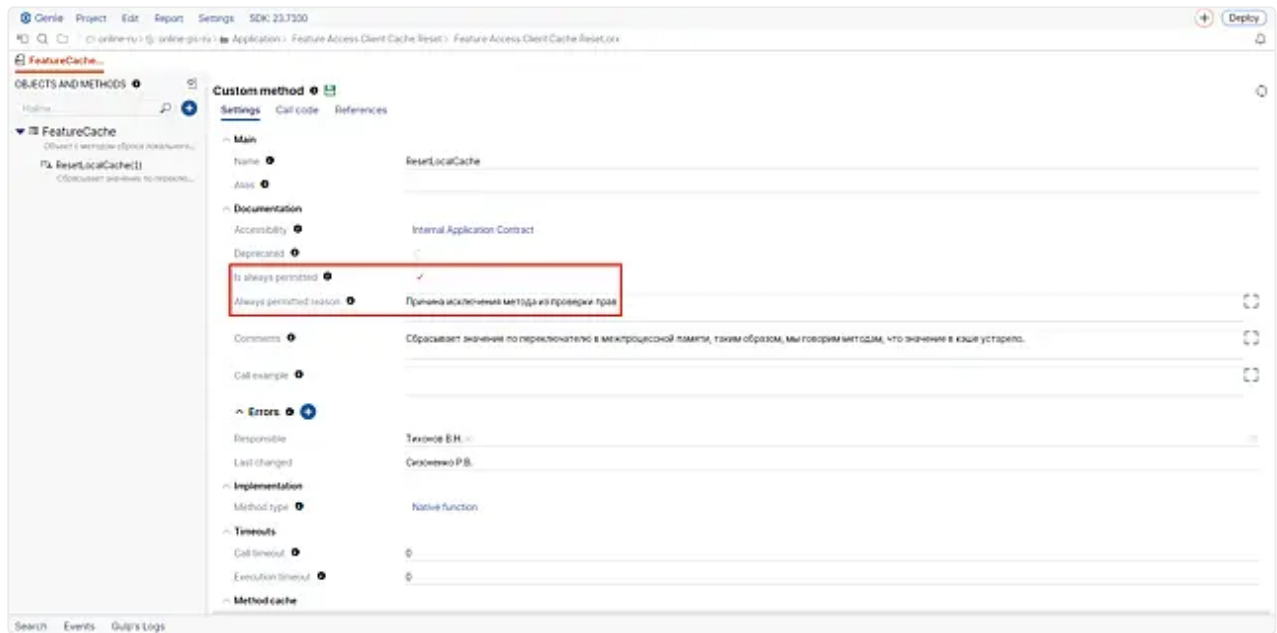
Добавление метода в список исключений

Для некоторых методов не требуется проверка прав. В этом случае необходимо указать в настройках метода причину исключения из проверки.

Обычно в список исключений добавляются методы, которые не работают с конфиденциальными данными или должны быть доступны любому пользователю до аутентификации.

Добавить метод в список исключений проверки прав можно следующими способами:

- В [Genie](#) с помощью отметки флага **Is always permitted** в настройках метода (рекомендуемый способ). Затем необходимо указать в поле **Always permitted reason** причину, по которой права на метод не проверяются.



- С помощью файла AllowMethods.txt (для [легкой системы прав](#)).
- С помощью файла exclude.rights (для [полной системы прав](#)).